CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: RestoRadar

Prepared By: Chan, Lance Lemmor B.

Version: 1.2

Date: 02/18/2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	02/02/2026	Lance Lemmor Chan	Initial draft	Draft
1.2	02/18/2026	Lance Lemmor Chan	Specified the features	Review
0.3	[Date]	Lance Lemmor Chan	Updated database design	Review
0.4	[Date]	Lance Lemmor Chan	Added UI/UX designs	Review
0.5	[Date]	Lance Lemmor Chan	Incorporated feedback	Revised
0.6	[Date]	Lance Lemmor Chan	Final review and corrections	Final
1	[Date]	Lance Lemmor Chan	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY..... 4

1.0 INTRODUCTION..... 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION..... 5

3.0 NON-FUNCTIONAL REQUIREMENTS..... 9

4.0 SYSTEM ARCHITECTURE..... 11

5.0 API CONTRACT & COMMUNICATION..... 12

6.0 DATABASE DESIGN..... 17

7.0 UI/UX DESIGN..... 18

8.0 PLAN..... 21

EXECUTIVE SUMMARY

1.1 Project Overview

RestoRadar is a location-based application designed to help users discover, explore, or review food establishments in their area. By aggregating real-time geolocation data, user-generated reviews, and detailed price information, the app aims to simplify decision-making for diners and enhance visibility for local establishments.

1.2 Objectives

1. Develop a fully functional app with real-time geolocation data, user reviews and ratings, detailed price ranges, and customizable profiles.
2. Implement a three-tier architecture using Spring Boot (backend), React (Web), and Android.
3. Incorporate REST APIs for communication between client and server.
4. Design a responsive user interface that works consistently across Web and Mobile platforms.
5. Deploy all system components to production-ready environments.

1.3 Scope

Included Features:

- User registration with OAuth (Google, X, etc.)
- Interactive map with views of food establishments
- Filter views to show price ranges (~~₱~~, ~~₱₱~~, ~~₱₱₱~~)
- User reviews and ratings
- Filter views via distance (< 5km)
- Photos

Excluded Features:

- Reservation to restaurants
- Ordering of food
- Social Media Integration
- User private messaging

- Analytics Dashboard

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for RestoRadar. It provides detailed specifications, architectural decisions, database design, and an implementation roadmap to manage the development of the app.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RestoRadar
Domain: Location-based
Primary Users: Tourists and Locals

Problem Statement: Tourists often have difficulty finding good or local food establishments because locals often do not have a platform for others to notice.
Solution: A location-based app that allows locals to post their establishments for others to check out and allows tourists to learn about nearby food establishments.

2.2 Core User Journeys

Journey 1: First-time Customer Purchase

1. User visits web or mobile application
2. Clicks “Sign Up” and creates an account via OAuth
3. Grants location permission
4. Sees interactive map with nearby restaurants
5. Applies filter (distance or price range)
6. Clicks a restaurant marker to view details
7. Reads reviews and ratings
8. Posts a review and rating
9. Saves restaurant to Favorites

Journey 2: Returning User Quick Search & Review

1. User logs in with existing credentials

2. Uses search bar to find a specific restaurant or cuisine
3. Filters results by price range
4. Selects a restaurant from the list
5. Views updated ratings and photos
6. Edits or deletes previous review
7. Adds another review after new visit
8. Logs out

Journey 3: Administrator Restaurant Management

1. Admin logs in with admin credentials
2. Navigates to Admin Dashboard
3. Reviews pending restaurant submissions
4. Approves or rejects new listings
5. Edits restaurant details if needed
6. Deletes inappropriate reviews
7. Manages user accounts (suspend/remove if necessary)
8. Logs out

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration and login (OAuth – Google, etc.)
2. Location detection (real-time geolocation)
3. Interactive map displaying nearby food establishments
4. View establishment details (name, description, location, price range)
5. User reviews and ratings system
6. Filter by distance (e.g., <5km)
7. Filter by price range (~~₱~~, ~~₱₱~~, ~~₱₱₱~~)
8. REST API integration between frontend and backend
9. Responsive UI (Web + Mobile support)

10. Basic database for users, establishments, and reviews

SHOULD HAVE

1. Search by restaurant name or cuisine
2. Save/Favorite establishments
3. Edit/Delete own reviews
4. Profile management (update profile info)
5. Admin approval for new establishment listings
6. Basic content moderation (report reviews)

COULD HAVE

1. Photo uploads for establishments
2. Photo uploads for reviews
3. Sorting (highest rated, nearest, most reviewed)
4. Dark mode UI
5. Basic analytics for establishment owners (views, rating average)
6. Push notifications (new nearby restaurants)

WON'T HAVE

1. Restaurant reservations
2. Food ordering and payment system
3. Delivery tracking
4. Social media integration
5. Private messaging between users
6. In-app chat support
7. Advanced business analytics dashboard

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Login, Registration
- **Authentication:** OAuth (Google, etc.)

- **Fields:** Email, Password, Confirm Password
- **Functions:** Register/Login, Logout, Role-based access (User / Admin)
- **API Endpoints:** POST /auth/login, POST /auth/register, GET /users/{id}

Feature: Restaurant Catalog

- **Screens:** Restaurant Listing, Restaurant Detail
- **Display:** Map view + Grid/List view, restaurant name, rating, price range (₱–₱₱₱), distance
- **Search:** By restaurant name or cuisine
- **Filters:** Distance (<5km), Price range (₱, ₱₱, ₱₱₱)
- **API Endpoints:** GET /restaurants, GET /restaurants/{id}, GET /restaurants/search, GET /restaurants/nearby?lat={lat}&lng={lng}&radius=5
- **Admin Functions:** POST /restaurants, PUT /restaurants/{id}, DELETE /restaurants/{id}

Feature: Interactive Map

- **Screens:** Map View
- **Display:** User current location, Restaurant markers, Marker popup (name, rating, price)
- **Functions:** Real-time geolocation detection, Click marker → open Restaurant Detail
- **Permissions:** Location access required
- **API Endpoints:** GET /restaurants/nearby

Feature: Reviews & Ratings

- **Screens:** Restaurant Detail (Reviews Section), Add/Edit Review
- **Display:** Star rating (1–5), Review text, Username, Date posted
- **Functions:** Add review, Edit own review, Delete own review, Report review
- **API Endpoints:** GET /restaurants/{id}/reviews, POST /restaurants/{id}/reviews, PUT /reviews/{id}, DELETE /reviews/{id}

Feature: User Profile

- **Screens:** Profile View, Edit Profile

- **Display:** Username, Profile picture, List of user reviews, Saved/Favorite restaurants
- **Functions:** Update profile info, Save/Favorite restaurant
- **API Endpoints:** GET /users/{id}, PUT /users/{id}, GET /users/{id}/favorites, POST /users/{id}/favorites/{restaurantId}

Feature: Admin Panel

- **Screens:** Admin Dashboard
- **Functions:** Approve new restaurant listings, Delete inappropriate reviews, Manage users
- **Access Control:** Admin role required
- **API Endpoints:** GET /admin/restaurants/pending, PUT /admin/restaurants/{id}/approve, DELETE /admin/reviews/{id}, DELETE /admin/users/{id}

2.5 Acceptance Criteria

Feature: User Registration & Login (OAuth)

AC-1: Successful OAuth Registration

- **Given** a new user is on the login page
- **When** the user selects "Sign in with Google" and authenticates successfully
- **Then** the system creates a new user account
- **And** redirects the user to the homepage

AC-2: Existing User Login

- **Given** a registered user
- **When** the user logs in via OAuth
- **Then** the system authenticates the user
- **And** loads their profile and preferences

AC-3: Unauthorized Access Restriction

- **Given** a non-authenticated user
- **When** they attempt to post a review
- **Then** the system prompts them to log in

Feature: Location Detection & Map Display

AC-4: Location Permission

- **Given** a logged-in user
- **When** the app requests location access
- **Then** the system detects the user's real-time location

AC-5: Display Nearby Restaurants

- **Given** location is available
- **When** the map loads
- **Then** restaurants within the default radius (e.g., 5km) are displayed

AC-6: Marker Interaction

- **Given** restaurants are displayed on the map
- **When** the user clicks a marker
- **Then** a preview appears
- **And** selecting it opens the restaurant detail page

Feature: Restaurant Details

AC-7: View Restaurant Information

- **Given** a user selects a restaurant
- **When** the detail page loads
- **Then** the system displays:

    - Name
    - Description
    - Address
    - Price range
    - Average rating
    - Reviews

Feature: Filtering

AC-8: Filter by Distance

- **Given** restaurants are displayed
- **When** the user selects distance filter (<5km)
- **Then** only restaurants within 5km are shown

AC-9: Filter by Price Range

- **Given** restaurants are displayed
- **When** the user selects a price range (₱ / ₱₱ / ₱₱₱)
- **Then** only restaurants matching the selected range are displayed

Feature: Reviews & Ratings

AC-10: Add Review

- **Given** a logged-in user on a restaurant page
- **When** the user submits a rating (1–5 stars) and review text
- **Then** the review is saved to the database
- **And** appears in the review list

AC-11: Edit Review

- **Given** a user has an existing review
- **When** the user edits and submits changes
- **Then** the updated review is saved
- **And** reflects immediately on the page

AC-12: Delete Review

- **Given** a user owns a review
- **When** they choose delete
- **Then** the review is removed from the system

Feature: Favorites

AC-13: Save Restaurant

- **Given** a logged-in user
- **When** the user clicks “Save” on a restaurant
- **Then** the restaurant is added to the user's favorites list

AC-14: View Favorites

- **Given** a user has saved restaurants
- **When** the user opens their profile
- **Then** the favorites list is displayed

Feature: Admin Management

AC-15: Approve Restaurant

- **Given** an admin is logged in
- **When** a restaurant is pending approval
- **Then** the admin can approve it
- **And** it becomes visible on the public map

AC-16: Remove Inappropriate Review

- **Given** an admin views reported reviews
- **When** the admin deletes a review
- **Then** the review is removed from the system

AC-17: Role-Based Access

- **Given** a normal user
- **When** they attempt to access admin routes
- **Then** access is denied

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Nearby restaurants shall load on the map within ≤ 3 seconds after location detection.

- Restaurant search results shall be returned within ≤ 2 seconds.
- Support 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- The system shall use secure OAuth authentication.
- All communication between client and server shall use HTTPS (TLS encryption).
- If any credentials are stored, they must be securely hashed and never stored in plain text.
- Admin functions shall only be accessible to users with admin roles.
- All user inputs shall be validated to prevent SQL injection, XSS, and other injection attacks.

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

- The UI shall function properly on desktop, tablet, and mobile screen sizes.
- A first-time user shall be able to discover a restaurant within **3 clicks/taps** after login.
- The application shall use readable fonts, sufficient color contrast, and intuitive navigation.
- Clear error messages with recovery options

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram

```
┌──────────────────────┐   ┌──────────────────────┐
│  WEB CLIENT          │   │ MOBILE CLIENT        │
│   (React)            │   │ (Android)            │
│  • TypeScript        │   │ • Kotlin             │
│  • Tailwind CSS      │   │ • Compose            │
└──────────┬───────────┘   └──────────┬───────────┘
           │                          │
           └──────────┬───────────────┘
                      │
                      ▼
           ┌──────────────────────┐
           │  BACKEND API         │
           │  (Spring Boot)       │
           │  • Java 17           │
           │  • Spring Security   │
           │  • JPA/Hibernate     │
           └──────────┬───────────┘
                      │
                      ▼
           ┌──────────────────────┐
           │  DATABASE            │
           │  (PostgreSQL)        │
           │  • Users table       │
           │  • Products table    │
           │  • Orders table      │
           │  • Cart items table  │
```

└──────────────────────────┘

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.marketplace.com/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

```
json
{
  "success": boolean,
  "data": object|null,
  "error": {
    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Product Endpoints

GET /products

Query: ?page=1&limit=20&search=keyword&category=electronics

Response: {products: [...], pagination: {page, limit, total, pages}}

GET /products/{id}

Response: {product: {id, name, description, price, stock, imageUrl, category}}

POST /products (Admin only)

Body: {name, description, price, stock, imageUrl, category}

Response: {product: {...}}

PUT /products/{id} (Admin only)

Body: {name?, description?, price?, stock?, imageUrl?, category?}

Response: {product: {...}}

Cart Endpoints

GET /cart (Authenticated)

Response: {cart: {id, items: [...], total, itemCount}}

POST /cart/items

Body: {productId, quantity}

Response: {message: "Added to cart", cartItem: {...}}

PUT /cart/items/{itemId}

Body: {quantity}

Response: {message: "Cart updated", cartItem: {...}}

DELETE /cart/items/{itemId}

Response: {message: "Removed from cart"}

Order Endpoints

POST /orders

Body: {shippingAddress: {fullName, address, city, zipCode, country}}

Response: {order: {id, orderNumber, total, status, items: [...], createdAt}}

GET /orders

Response: {orders: [...]}

GET /orders/{id}

Response: {order: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

```
json
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2024-01-28T10:30:00Z"
}

{
  "success": false,
  "data": null,
```

```
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```


Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD

```
USERS (1) ——— (1) CARTS (1) ——— (*) CART_ITEMS (*) ——— (1) PRODUCTS
   |                    |
   |                    |
  (*)                  (*)
```

```
  │                    │
  │                    │
ORDERS (1) ─────── (*) ORDER_ITEMS (*) ──────┘
```

Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[🏠 Home] [🔍 Search] [🛒 Cart] [👤 Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements